

The Impact Of Address Mapping Scheme And Scheduling Policy On DRAM Power And Performance

Mclauchlan Lifford

Department of Electrical Engineering
and Computer Science
Texas A&M University
Kingsville, Texas, US

Nekovei Reza

Department of Electrical Engineering
and Computer Science
Texas A&M University
Kingsville, Texas, US

Hicks David

Department of Electrical Engineering
and Computer Science
Texas A&M University
Kingsville, Texas, US

Fleku Dennis Elorm

Department of Electrical Engineering
and Computer Science
Texas A&M University
Kingsville, Texas, US

Rahman Ashfaqur

Department of Electrical Engineering
and Computer Science
Texas A&M University
Kingsville, Texas, US

Abstract—Dynamic Random Access Memory (DRAM) memory controllers manage the movement of data into and out of DRAM devices while maintaining compliance with DRAM device specific electrical characteristics, timing characteristics, and other system specific features. DRAM controllers serve as an important component of the DRAM systems. Studies [1] [2] [3] in recent times have focused on ways to improve various aspects of DRAM memory controller design in order to increase overall DRAM memory system performance with maintaining optimal overall DRAM power consumption.

In the same vein, this study provides an insight into the different design considerations adopted in modern DRAM memory controllers (proposed in academia/industry) from a performance and power consumption view point. It takes a step further to adopt a first order approximation model to compare the collective effect of the memory controller's address mapping scheme and scheduling policy on DRAM performance and power efficiency. It was evident through a series of simulation tests that a better choice of address mapping scheme and scheduling algorithm can optimize memory performance and reduce overall power consumption of DRAM memory system.

Index Items—dynamic, random, memory, central, processing unit, synchronous, bandwidth, operation, dual in-line, latency, power, row, column, bank, rank, channel, power, performance.

I. INTRODUCTION

Computer system performance has considerably improved mostly in the areas of processor speed and power capabilities over the years. This has been influenced by the utilization and adoption of efficient multiprocessor design schemes for the heterogeneous needs of modern system applications, with the performance of main memory growing at a much lower rate. With the advancement in semiconductor process technologies and scaling, studies [1] [2] have shown that microprocessor performance has steadily increased at a rate of 60% per year, while the memory subsystem lags at a yearly growth of 10%, depicted in Fig. 1. An area of major concern for memory system designers has been the heterogeneous nature of system level applications. The shift of trend to support concurrent multi-core systems and workloads [3] has presented varying latency and timing requirements for computing systems as a whole and main memory an important sub-component.

To achieve low memory access latency and improve memory bandwidth, modern memory systems like the Double Data Rate DRAM (DDR-3, DDR-4, etc.) [4] exploits the hierarchical structure of main memory (DRAM) to introduce parallelism. Performance driven approaches from an architectural standpoint have been considered for memory controller design to aid data transfer between CPU and main memory for the different system level application requirements. Design aspects including, but not limited to, the address mapping scheme, row-buffer management policy, transaction and command queue organization, and scheduling algorithms of memory controller are continually being exploited to improve the latency, bandwidth and power characteristics of the whole DRAM system.

This study would provide an account of design trends adopted in the above mentioned areas in modern DRAM memory controller designs. Further, an analyses of the effect of the interplay between the memory controller's address mapping and scheduling policy on DRAM power and performance was performed using custom memory traces, select address mapping schemes and scheduling policies.

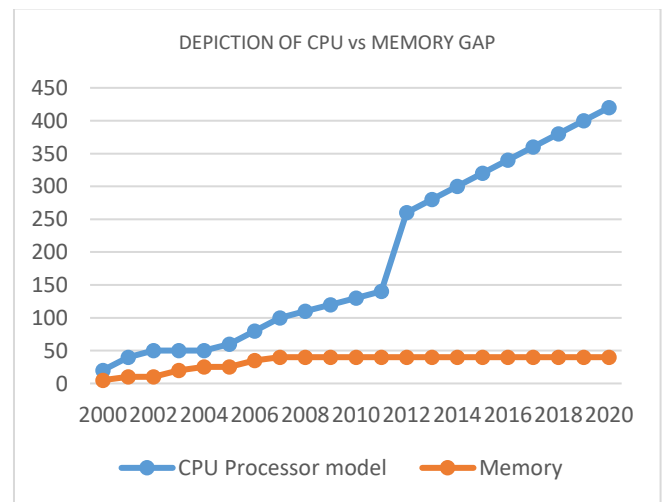


Fig. 1. Yearly improvement of processor and DRAM memory speeds, for a time period of two decades. (depiction of CPU vs memory gap)

A first order approximation model in the form Average Read Latency per Memory Operation (ARLMO) and Average Power consumed per Active Cycle (APAC) was utilized for simplification of discussion to study the effects of these two memory controller design parameters on overall DRAM power and performance.

The memory performance toll on power utilization behavior was clearly captured in the test results with the ARLMO and APAC metric values indicative of the power and performance inverse behavior. In addition, it was discovered that a good selection of DRAM memory controller address mapping scheme and scheduling policy can greatly improve both the performance and power requirements of DRAM systems, a baseline for conducting studies of other memory technologies.

II. RELATED WORK AND MOTIVATION

This section provides an overview of some memory controller design schemes proposed in both academia or industry and how they contribute to improve memory performance.

A. Row Buffer Management Policy (Open, Close Page Policy)

System level applications can be temporal or spatial based [12]. Temporal locality allows memory request indexing to the same memory location. This is in contrast to spatial locality principle which can guarantee previous and current accesses to an adjacent memory block but not the same location. These two principles provide the basis for design considerations employed in DRAM memory controllers for memory latency optimization.

Modern DRAM memory controllers offer a dynamic blend of open-page and close-page policy. Studies [14] [15] on performance and power consumption impact of row buffer page policy on memory system indicates that the optimality of a row buffer management policy type depends on the memory access rate and the access pattern. This is achieved by the combined dependence on memory request history and timers to match the dynamic system access patterns to a well suited row buffer policy in modern DRAM controllers.

The timer action operates by an initial setting to a specific value at each row buffer policy change and counts down every clock cycle. It changes to the opposite row-buffer policy depending on the current policy at the end of the current count and resets the time counter based on the page policy metric value. It is worth-noting that row-buffer management policies can be applied on a channel-by-channel basis or bank-by-bank basis based on design requirements and a trade-off between design complexity and performance can be decided.

B. Address Mapping Scheme

The address mapping translation describes a mechanism used to segment physical address into indices (ID) of channel ID, rank ID, bank ID, row ID, and column ID. Correctly matching the run-time behavior of applications with the address mapping scheme of DRAM memory system can prevent the occurrence of bank conflicts scenarios. Bank conflicts degrade DRAM memory performance. An effective address mapping scheme tends to prevent bank conflicts and

increase parallelism in the memory system. A choice of a particular address mapping scheme can be decided jointly with the row-buffer management policy of the DRAM memory controller.

Conventional system designs shown in Fig. 2. with processor operating in virtual address space have the translation look-aside buffer (TLB), housed in the memory management unit (MMU), which maps the virtual address to physical address without prior knowledge of mapping scheme in the DRAM system. Study [17] proposed the use of a section of the address space as ‘shadow addresses’ (unused addresses) while combining a novel virtual-to-physical address translation scheme and an intelligent memory controller for dynamic address space indexing. This is achieved by integrating the DRAM memory controller within the TLB for the DRAM system.

The hierarchical nature of DRAM memory system presents an avenue for the review of address mapping schemes from a memory parallelism view point. The channel provides the highest level of parallelism from a top-to-bottom analysis of the memory hierarchy. High switching penalty in mostly Double Data Rate (DDR) memory systems make rank level parallelism undesirable. The banks in ranks of DRAM memory provides better efficiency in mapping consecutive read accesses to bank arrays than the ranks since idle switching cycles are not needed. Advanced memory controllers with read-write request intermixing defer write requests for scheduling and more often have bank addresses mapped lower than rank addresses to exploit locality in memory request sequences. In the case of rows, conventional DRAM memory allows an active row per bank (though modern DRAM memories permit more than one row per bank active) which forces mapping of memory accesses to different rows of the same bank. As a result, row addresses are mapped to higher address ranges to prevent bank conflict situations.

Column addresses can be appropriately mapped for the proper operation of either open-page or close-page systems.

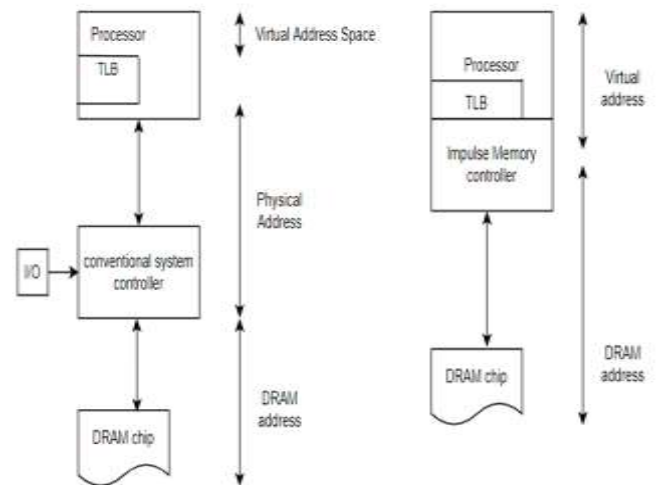


Fig. 2. Conventional System Architecture vs Impulse Memory System Architecture

For open-page, column addresses are mapped to lower address bits of physical addresses to maximize hit rates. But in contrast to close-page DRAM systems, a portion of the column index is mapped to lowest address while the remainder of the column address is mapped to the high address ranges.

The Bank-Row-Column (BRC) and the interleaved Row-Bank-Column (RBC) are examples of common addressing schemes. The latter scheme, also called Page Mode or Bank Interleaving [19], brings the upper bank bits of the address down between the row and the column bit to ensure a better bank utilization. Also, other sophisticated mapping schemes such as the Permutation-Based Page Interleaving Technique [20] in Fig. 3. re-maps potential “bank-conflict” memory accesses to different banks by “xoring” the bank bits of the memory address with selected row bits to exploit bank-level parallelism.

C. Transaction Scheduling and DRAM Command Policies

The idea of memory access scheduling is that in order to fully utilize memory bandwidth, reduce access latency, improve overall system performance, and re-order the memory access requests under the premise of meeting the memory timing requirements.

DRAM memory requests scheduling is a conundrum, requiring a balance between obeying access scheduling constraints, prioritizing memory requests properly, and adapting to a dynamically changing memory reference stream. The memory controller in modern DRAM systems can accept memory requests from multiple request agents and provides the arbitration interface to choose which requests are admitted into the memory controller. An arbiter controls the flow of traffic between requestors and shared memory. Different memory scheduling techniques such as round robin, first ready - first come first served (FR-FCFS) or priority based schemes have been proposed to increase the overall memory system performance by maximizing row-buffer hit rates, bank-level parallelism and fairness.

In round robin, each system requestor is allocated specific amount of clock cycles to access memory in a cyclic pre-defined order. The round robin approach provides a fair access to each memory requestor and prevents against starvation of requestors with low-to-partially filled buffers.

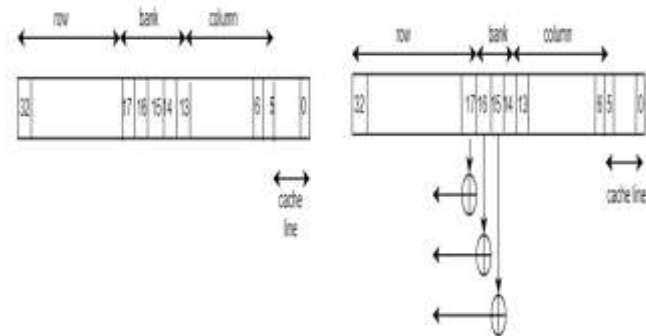


Fig. 3. Depiction of Row Bank Column Address Mapping and XOR Address Mapping [18].

The First Ready - First Come First Serve (FR-FCFS) [21] scheme targets execution of out-of-order DRAM operations to enhance overall system performance. The arbiter keeps track of which memory agents asserts a READY signal first. FR-FCFS mechanism is one of the commonest DRAM scheduling algorithms and uses two ordering mechanisms to exploit the row-buffer locality feature. It uses a conservative re-ordering method that prioritizes oldest pending memory transactions over other transactions even if the scheduling of the oldest pending transaction degrades system performance. But the aggressive reordering mechanism would avoid system performance degradation by reordering pending transactions in a way that maximizes the row-buffer hit ratio. The FR-FCFS operates on three main principles:

1. Ready-First: prioritize ready commands over unready ones.
2. Column-First: prioritizes column accesses, read and write commands over row accesses.
3. Oldest-First: prioritizes older memory transactions over recent ones.

FR-FCFS performs well on single-threaded systems [6] but poorly on multi-thread systems.

Another scheduling approach is the parallelism-aware batch scheduling (PAR-BS) [6] which uses batch execution by limiting the numbers of requests per bank to increase system performance. The idea is to group pending requests in a buffer with an assigned marking-cap. Another variant of the PAR-BS is the parallelism-aware within-batch scheduling which prioritizes within-batch requests, row-hit-first requests, higher-rank first requests, and oldest requests.

An application-aware scheduler, Staged Memory Scheduling (SMS) [22], decouples memory requests scheduling into three (3) tasks: batch formation, batch scheduler, and DRAM command scheduler. This scheduling setup is used in integrated CPU-GPU systems.

Memory access command scheduling can be viewed in terms of static, semi-static and dynamic behavior. [23] A static DRAM command scheduling algorithm is proposed using a local search method. Statically scheduled DRAM controllers requires foreknowledge of memory access pattern at the application-level to efficiently schedule commands. This approach of executing SDRAM commands computed at design time is unsuitable for real-time systems with unpredictable workloads though its use in reconfigurable devices is well-placed. Semi-static based DRAM controllers [24] fail to provide the lowest possible execution time of memory requests of both real-time and conventional applications and they are limited in design structure. The drawback of this mechanism is its inability to cope with changing transaction sizes. Study [25] noted that the idealness of DRAM scheduling algorithm depends not only on the efficiency of DRAM scheduling policy but also on the application level requirements.

On the other hand, dynamically scheduled memory controllers schedule DRAM commands at execution time based on incoming memory request stream. Such controllers are designed to serve high-performance systems providing them with the flexibility to match run-time memory requests timing needs. Their operation mechanism can involve high priority handling of requests that are directed to open rows in the memory bank’s sense amplifier [6] [26] [27], or that

which matches the current direction of the data path [28] [29] [30]. An attempt to capture the unpredictable nature of systems makes these dynamic based controllers have complex hardware implementations and will require much design time and verification efforts.

The idea of write caching assigns a level of criticality to memory requests by treating write requests as non-critical in terms of latency while read requests are handled as latency critical. Write requests are deferred while read requests are scheduled for memory access. A key condition of write caching is the maintenance of the program order correctness and the existence of architecture support. DRAM devices typically schedule poorly back-to-back read and write requests, resulting in a column read command following a column write command. This situation introduces a high overhead for data bus switching so modern DRAM memory controllers have write caching capability to schedule buffered write requests as a single burst access.

D. Transaction & Command Queue Organizations

Current trends in request queue organization support the conversion of DRAM memory transactions into DRAM commands, which are stored in a single or multiple queue structure. The queue organization in DRAM memory controllers provide benefits including priority based scheduling of DRAM commands, bank resource consideration during scheduling, and pipelined execution of DRAM commands. One queue organization commonly used in DRAM memory controllers is the per-bank per queue organization Fig. 4. This type of queue structure assigns equal priority to different bank request queues and processes request on a bank-by-bank basis. Memory transaction commands are directed into respective queues based on their bank address. The open-page DRAM memory controller utilizes the per-bank configuration to increase row buffer hits since multiple requests can be sent only to the same open row. Once a bank's request queue gets exhausted a pre-charge command is issued and request control switches to next bank in the priority list.

Round-robin mechanism, a cyclic bank-to-bank rotation command scheduling scheme, can be used with the per-bank queuing structure. In the open-page DRAM memory controller implementation, bank conflict overhead can be hidden if there is enough requests to queues serving other banks. Close-page based DRAM controllers using round-robin scheduling to schedule cache lines are prone to increased spatial distance between requests and starvation of requests. Combining the round-robin approach with weights assigned to banks, depending on the number of requests in its request queue, can reduce request starvation.

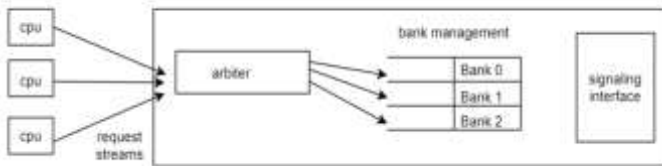


Fig. 4. Per bank queue organization structure

III. METHODOLOGY

A. Motivation

The research activities [6] [31] [32] sought to ascertain the impact of application based optimizations or memory controller scheduling algorithms on DRAM memory system performance through the comparison of performance metrics such as latency, and power consumption. In research [6], bank-level parallelism was utilized in a batch scheduling algorithm to schedule memory requests in batches. The study [31] proposed a dynamic multiple address mapping scheme and experimented its impact on DRAM performance. The study [32] sought to verify the effect of DRAM memory controller scheduling policy on DRAM's energy utilization and performance. It is worth noting a trend in the past studies which verifies individually either the impact of the controller's address mapping scheme or its scheduling algorithm on DRAM performance. The following questions arises: What if there is a good address mapping scheme that enhances parallelism but the scheduling algorithm poorly schedules DRAM commands? Does a similar question arise for the opposite scenario?

As a result, this research work was poised to find out the effect of the interplay between DRAM memory controller's addressing mapping and the scheduling algorithm on DRAM performance and power. It relied on adjusted metrics that measured the memory latency, and power utilization to perform its analysis.

B. Simulation Model

In order to be able to undertake this work, there is a need for a cycle-accurate simulator that can correctly model the DRAM controller parameters. Due to the complexity of a DRAM system architecture, device timing constraints, configurations, and memory controller protocol, most of the existing simulators [33][34] do not fully capture detailed architectural features of memory systems and thus model memory access as a fixed latency task. Thus, having a representative memory-controller performance model is vital. DRAMSim2, a cycle-accurate simulation tool, uses an object oriented approach in C++ to model DDR2/3 memory system including the memory controller. The DRAMSim2 core can be instantiated in a single object "MemorySystem". A MemorySystem object is composed of two ini files: a device ini file and a system ini file. A device ini file contains DRAM device specific parameters such as the timing constraints and power consumption of the DRAM device. The system ini file consists of parameters that are independent of the actual DRAM device including but not limited to: the number of ranks, the address mapping scheme, debug options, row buffer policies, memory controller queue structures, and other details. Also the DRAMSim2 allows easy integration with any type of front end driver such as a trace reader, a cycle accurate CPU simulator such as MARSSx86.

DRAMSim2 also uses Micron [35] power model to compute power consumption using state transitions for each bank. This comes with an added power saving mode feature that can be enabled in the ini file. Event based simulator [36] was designed to provide a more accurate model of DRAM memory by modeling memory using state transitions.

DRAMsim3 [38], an upgrade of DRAMsim2, provides the ability to model DRAM systems by specifying DRAM device parameters such as number of rows, columns, and other system parameters contained in the “config/DDR*” (DDR* relating to specific protocol) directory. DRAMsim3 supports DDR4 SDRAM protocol which was the target DRAM technology for this thesis. A software architecture of DRAMsim3 is shown in Fig. 5 below.

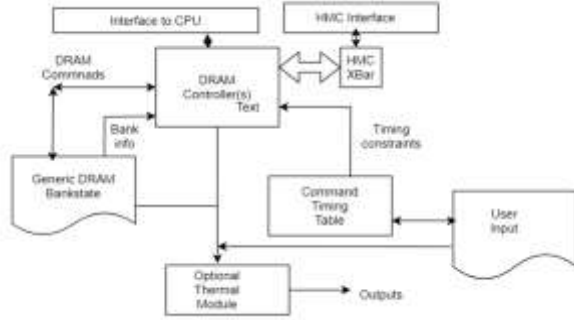


Fig. 5. Software Architecture of DRAMsim3

C. Experimental Setup

Trace driven analysis and CPU system integration are two commonly used front-end setup adopted in the computer architecture simulation community. Although CPU integrated front-end drivers provide better modeling of system behavior at run-time, the trace driven approach was used in this research since it was sufficient to verify the memory controller design changes and run-time behavior. Traces are easy to produce by recording real CPU memory request patterns intensity similar to STREAM suite (for sustained bandwidth testing), GUPS (random requests), and high performance conjugate gradients (HPCG-memory intensive workloads). But in the case of this study, custom trace files in Table I were generated for specific test scenarios using the “trace_gen.py” utility incorporated in the DRAMsim3 and pre-processed into a readable format.

It is ensured that each set of memory trace file has a READ: WRITE ratio close to 3:2. Also, the trace_gen.py utility provides switches to define inter-arrival time between requests, control READ:WRITE ratio and the number of streams (-i, -r, -n respectively). A sample of trace file format with random memory accesses is provided below:

TABLE I. TRACE DATA TYPES AND TEST CASES

Custom Trace Type	Test Scenario
Stream	Memory accesses to exploit locality
Random	Random memory accesses
Mix	Combination of stream and random traces
Batch_mix	Mix trace type with different timestamps to mimic batch scheduling
Mix_wcache	Mix trace type edited to mimic memory write caching behavior

```

0x195cd97d WRITE 90
0x9ab69a9a READ 95
0x498b140 READ 100
0x79897c55 READ 105
0x967128c7 READ 110
0x199b5424 READ 115
0xf8de5382 READ 120
0x8bca04e9 WRITE 125
0xb6a0b79c WRITE 130

```

In relation to the simulated DRAM device, DDR4 protocol configuration and timing parameters similar to ones published in both industry and research such as [39] [40] were adopted. Two memory configurations, 1-channel and 4-channel were considered for the simulation and analysis. Sample DRAM memory structure, timing constraints, system settings and power parameters contained in DRAMsim3 tools is provided in Tables II, III, IV and V. The “configs/” directory contains various DRAM specific protocol files in the DRAMsim3 tool.

TABLE II. DRAM MEMORY CONFIGURATIONS

No. of Bank group	Bank s/group	Rows	Device Width	Column	Burst Length (BL)
2	4	32768	16	1024	8

TABLE III. SAMPLE TIMING CONSTRAINTS OF SIMULATED DDR4 PROTOCOL

Timing	Cycles	Description
tCK	1.07	Clock latency
AL	0	Added latency to column accesses
CL	13	Column access latency
CWL	10	Column to Write access latency
RCD	13	Minimum time between row activation command and row buffer data availability
tRP	13	Interval between pre-charge command and next row opening
tRAS	32	Row activation to pre-charge command latency
tRFC	243	Time interval between refresh command and next valid command
tREFI	7285	Refresh inter-interval latency
tRPRE	1	Read to pre-charge command
tWPRE	1	Write to pre-charge command

tFAW	28	Minimum number of row activations per bank/rank basis
tWR	14	Added latency for write operation safe completion
tRTP	7	Read to Precharge command
tCCD_S	4	Minimum time between successive reads or writes

TABLE IV. DRAM SYSTEM SETTINGS

Parameters	Values
Channel_size	4096
Channels	1 (or 4)
Bus_width	64
Address_mapping	rochrababgco
Queue structure	PER_BANK
Row_buf_policy	OPEN_PAGE

TABLE V. SAMPLE DRAMSIM3 POWER MODEL

Power Parameters	Value (mW)
Voltage (VDD)	1.2
IDD0 (Operating Current)	65
IPP0 (Operating current)	3.6
IDD2P (Precharge power-down current)	27
IDD2N (Precharge standby current)	40
IDD4W (Operating burst write current)	220
IDD4R (Operating burst read current)	180

The address mapping scheme comes in different variations depending on intended design goals and the type of input stream. Normally the bank bits are mapped to the lower order bits of physical address and rows are mapped to higher order bits of physical address in order to prevent bank conflict cases and to exploit memory parallelism in open-page systems. Rather, in closed-page systems, the bank bits are mapped higher while the row bits are indexed to lower address bits. This address mapping scheme can be chosen in relation with

the row page policy to increase memory performance. Table VI presents the various DRAM memory controller address mapping schemes considered for this study. Originally, the DRAMsim3 tool is designed to schedule memory commands using the FR-FCFS scheduling policy. In order to create other scheduling policy types, the open-page & close-page row policies are varied with the queue structure types (PER_BANK or PER_RANK). The variant scheduling policies use a queue structure setting of PER_RANK.

TABLE VI. ADDRESS MAPPING SCHEMES

Scheme	Mapping
Scheme A	rochrababgco
Scheme B	chbabgracoro
Scheme C	chrababgcoro
Scheme D	rocorababgch
Scheme E	babgcochraro
Scheme F	cochrarobabg

TABLE VII. DRAM SCHEDULING POLICIES

Scheduling Policies
Open page
Close page
Open page variant
Close page variant

The first simulation test was undertaken with a trace input containing 5 stream memory accesses. This is primarily to test setup environment for simulation and obtain initial system variables. For the next simulation tests, each trace input type in Table I is tested against the various DRAM address mapping schemes in Table VI while maintaining a default DRAM memory scheduling policy (open page policy). The objective of these tests was to ascertain the “best” and “worst” address mapping schemes for each trace type under normal system conditions. The “best” DRAM memory controller address mapping scheme should reduce instruction execution latency which increases overall DRAM power utilization at an optimum level. The opposite scenario that results in increased memory access latency and less optimum power usage should hold for the “worst” case address mapping scheme.

A similar set of runs is conducted but this time around, the memory controller’s scheduling policy is varied while the memory address mapping scheme is kept constant for each trace type (scheme A). Its purpose was to find out the “best”

(optimum) and “worst” performing DRAM memory controller scheduling policies for each trace type. It was assumed that DRAM memory controller performance and power values should not be considerably affected across different setup environments since a static test environment was adopted to model the runtime operation.

Now, the study performed the next category of address mapping-scheduling tests in the order listed below. The Average Read Latency per Memory Operation (ARLMO) and Average Power per Active Cycle (APAC) performance metric values were recorded and used for analysis.

- 1) Best address mapping scheme – Best scheduling policy (BB)
- 2) Worst address mapping scheme – Worst scheduling policy (WW)
- 3) Best address mapping scheme – Worst scheduling policy (BW)
- 4) Worst address mapping scheme – Best scheduling policy (WB)

D. Performance Metrics

Different units of measure can be used to compute DRAM’s performance and power utilization: the number of read/write operations, memory access latency, bandwidth and total power consumed. DRAM memory access latency determines the number of cycles consumed by a read/write operation, measured in cycles per instruction (CPI). An average CPI in terms of total number of operations and cycles consumed provides a generalized value.

$$\text{CPU Time} = \text{Cycle Time} * \text{Instruction Count} * \text{Average CPI} \quad (1)$$

$$\text{Average CPI} = \text{Summation of CPI and frequency of memory request} \quad (2)$$

The ARLMO provides a better view of the amount of cycles consumed for each memory operation. Lower values of this metric signifies a better memory instruction latency. A speedup value is calculated by comparing previous and recent execution time values.

$$\text{ARLMO} = \text{Avg. Read Latency} / \text{Number of operations} * \text{frequency} \quad (3)$$

$$\text{Speedup} = \text{Execution timeold} / \text{Execution timenew} \quad (4)$$

Power utilization can be accessed in two scenarios: single bank access and parallel bank access.

$$\text{APAC} = \text{Total power consumed} / \text{Number of active cycles} \quad (5)$$

APAC metric gives an indication of DRAM power efficiency during DRAM active cycles. It can be viewed as the average power consumed for each memory read/write operation cycle. Active cycle is the number of cycles consumed when memory is operational. As a result, small values of the APAC metric value will be desirable for optimum DRAM operation. However, the DRAMsim3 provides the overall power usage

value after each simulation test which is then used to calculate the corresponding APAC metric value.

IV. ANALYTICAL RESULTS AND OBSERVATIONS

A. Recorded Data And Observations

As part of the trace driven approach for this work, simulations were performed using randomized, stream, a combination of random and stream traces and other derived custom memory traces. Results of the various simulation tests served as the primary data for analysis. DRAMsim3 provides a switch (-o) on the command line to direct output of simulation to any user-defined output directory (default is current directory). A directory called “outputs” was created in the home directory of DRAMsim3 tool and all simulation tests results were directed to this output folder.

The plan for this section is to conduct a series of experimental simulations, record results and examine the data on the basis of the impact of address mapping scheme and scheduling policy of the DRAM controller settings on DRAM power and performance. Though this goal is quite difficult to obtain in isolation considering the different DRAM parameter settings, configurations, protocol specific timing constraints and modelling capabilities of available simulation environments, this thesis tried to establish a trend using the power and performance metrics adopted.

Different custom trace files in Table I mimicking system-level application requirements (like locality and randomness with interleaved READ/WRITE mixes) were deemed enough to meet requirements of the experiment conducted.

Testing began by conducting a set of simulation runs (about 20) to check simulation setup correctness, determine “trace_gen.py” utility switch parameters (request inter-arrival time “-I”, read:write ratio “-RW”, and number of requests “-N”) and other system variables. The 1-channel DDR4 protocol configuration with Scheme A (rochrababgco) mapping and close-page policy are adopted for the initial simulation “test”. Results of 5 simulation runs detailing their respective average read request latency and average power were recorded. The corresponding ARLMO and APAC values were calculated using the recorded values. A breakdown of the test results over 100 cycles is provided in Table VIII.

The next simulation tests would help determine the “best” or “worst” DRAM controller address mapping schemes and scheduling policies for each trace input type. The “best” or “worst” address mappings for each trace input was selected by varying the various address mapping schemes while keeping the scheduling policy constant (default = open page policy). The reverse procedure was used to determine the “best - worst” DRAM controller scheduling policy but this time around the address mapping scheme was held default (Scheme A = rochrababgco). The 1-channel configuration was maintained for these best/worst mapping and scheduling policy “selection” set of simulation tests. The number of requests (“N”) contained in each trace file was 500, the setting for the “I” parameter was varied between 2 or 5 and a RW ratio parameter setting of 2 was selected for the trace_gen.py tool. The RW ratio setting of 2 on the

TABLE VIII. TEST RUN RESULTS

Tests	Average Read request Latency (cycles)	Average power (mW)	Average Read Latency per Memory Operation	Average Power per Active cycle (mW/cycle)
1	25.4	630.48	1.016	17.04
2	26	703.44	0.52	22.69
3	33.25	641.28	0.665	8.438
4	33.33	704.16	1.667	22.354
5	47.22	949.92	2.361	19.19

TABLE IX. BEST –WORST MAPPING FOR ADDRESS SCHEME & SCHEDULING POLICY

Trace	Address Mapping Scheme		Memory Scheduling Policy	
	Best case	Worst case	Best case	Worst case
Stream	Scheme A	Scheme B	Open page	Close page
Random	Scheme D	Scheme F	Close page	Open page
Mix	Scheme D	Scheme C	Open page	Close page variant
Batch_mix	Scheme F	Scheme B	Open page variant	Close page variant
Mix_wcache	Scheme A	Scheme B	Open page	Close page variant

“trace_gen.py” keeps read:write ratio approximately close to 3:2. Similarly, the results of a subset of the various simulation runs performed are shown in the subsequent tables X, XI, XII, XIII, and XIV (Appendix section).

Under this category of simulation tests, the address mapping “best-worst” pair was determined first. The open-page policy was maintained as default scheduling policy while the address mapping scheme was varied. The essence was to assume normal operating conditions for the scheduling policy for these best/worst set of runs. The best address mapping scheme should provide higher memory parallelism, reduce ARLMO and with an optimum APAC value. The reverse scenario holds for the “worst” mapping scheme. It was observed that Schemes A, D and F with recorded low ARLMO values maintained had row bits mapped higher than banks (and associated bank groups). Schemes B, C and E maintain an opposite bank-row mapping compared to schemes A, D & F, thus their higher ARLMO value. The power and performance inverse behavior was at play, higher performance (less memory instruction execution time

(ARLMO)) demands higher overall power utilization and vice-versa, can be seen across results of the different schemes.

There were instances where performance was traded for power during the “best-worst” pair address mapping selection process: evident in the random trace. The unpredictability of the request patterns for the random trace was reflected in the ARLMO and APAC values: close range of values for the different schemes as shown in Table X. This signifies that other schemes can also map this trace with minimal performance dip.

Batch_mix contained requests polled at a quicker rate in groups (batches). Schemes A, D and F, with low ARLMO value, have bank bits of memory address mapped lower than the row bits. Schemes B, C and E recorded high ARLMO and have bank bits of memory address mapped higher than row bits. Results of the various metric values are listed in Table XIII.

The mix_wcache trace is a variant of the mix trace with memory requests re-ordered to mimic write caching feature in DRAM memory systems. The write caching behavior allows READ requests to be serviced first before the WRITE requests increasing memory performance in terms of latency. This was reflected in the recorded low ARLMO value for mix_wcache trace compared to that of the mix trace. Another key observation is that both the stream trace and mix_cache trace recorded similar best mapping (Scheme A) and worst mapping (Scheme B) schemes.

Similarly the simulation test results to select the “best/worst” scheduling policy pair are provided in the Tables XV, XVI, XVII, XVIII, and XIX (Appendix section). The best scheduling policy recorded lower ARLMO values and optimal APAC value. The opposite analogy held for the “worst” case scheduling policy. A default address mapping scheme (scheme A) was maintained for all simulation tests to assume normal operational mode for the address mapping scheme. Most of the traces recorded low ARLMO values for the open page policy (giving an indication of row buffer hits) while high ARLMO values for close page policy particularly for the stream, mix and mix_wcache traces.

A final breakdown of the best and worst address mapping and scheduling policy for the each trace file after the various simulation tests is provided in Table IX.

To determine the speed-up obtained by correctly pairing of the address mapping scheme and scheduling policy to the memory request traces, the pairings below were used as the memory controller settings for both the 1-channel and 4-channel configurations.

- 1) Best address mapping scheme – Best scheduling policy (BB)
- 2) Worst address mapping scheme – Worst scheduling policy (WW)
- 3) Best address mapping scheme – Worst scheduling policy (BW)
- 4) Worst address mapping scheme – Best scheduling policy (WB)

The other settings captured are: the number of requests (n=30000), number of cycles increased to 100,000 and I parameter varied between 2 or 5.

The results for each BB, WW, BW and WB tests were presented on the clustered bar graph, ARLMO (on channel basis) vs APAC (on channel basis) are illustrated side-by-side

for BB, WW, BW pairings. The Power and Performance gain/loss in terms of ARLMO and APAC metric values for each memory are readily depicted and comparisons can be visually made effectively. The next group of figures presents the clustered bar representations for each trace file simulated using the 1-channel and 4-channel memory configurations. Generally, it was observed that the BB (best mapping – best scheduling policy) category recorded better ARLMO and APAC values for most traces for both 1 and 4 channel configurations. Overall, the 4-channel configuration recorded lower ARLMO value than the 1-channel configuration due to increased parallelism in the 4-channel configuration. The WW category recorded high ARLMO and APAC values for most of the trace tested. BW and WB categories represented scenarios of either having an appropriate address mapping or scheduling policy which can improve or deteriorate DRAM performance depending on the type of memory trace. Results are shown for the 4-channel configuration in Fig. 6, 7, 8, 9, and 10 below.

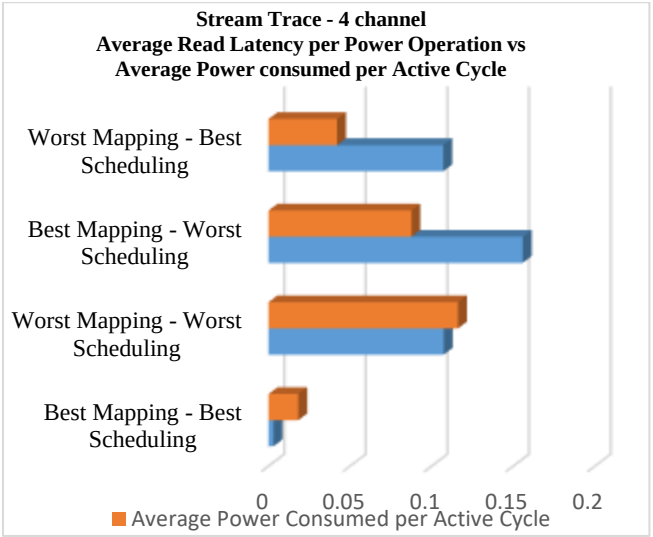


Fig. 6. ARLMO vs APAC stream trace plot (4 -channel).

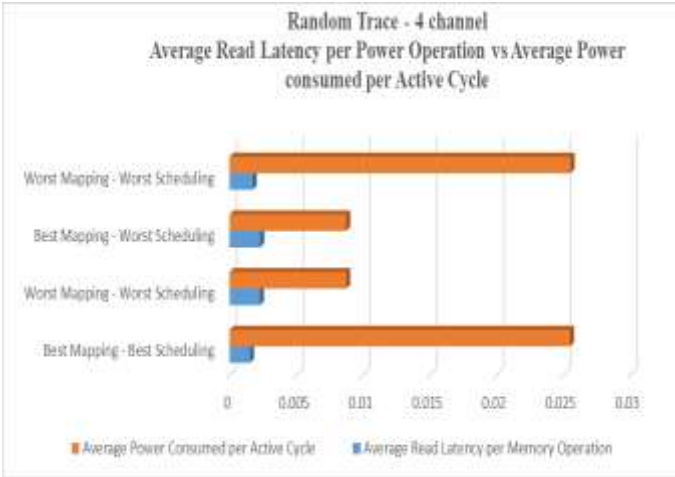


Fig. 7. ARLMO vs APAC random trace plot (4 -channel).

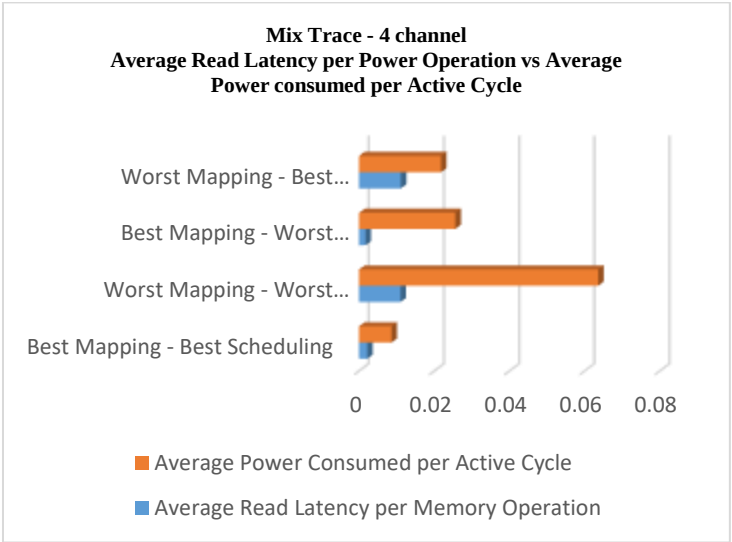


Fig. 8. ARLMO vs APAC mix trace plot (4 -channel).

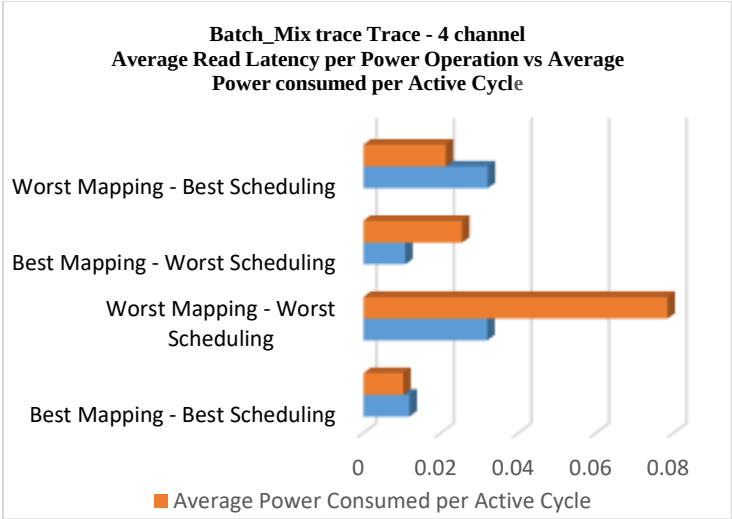


Fig. 9. ARLMO vs APAC batch_mix trace plot (4 -channel).

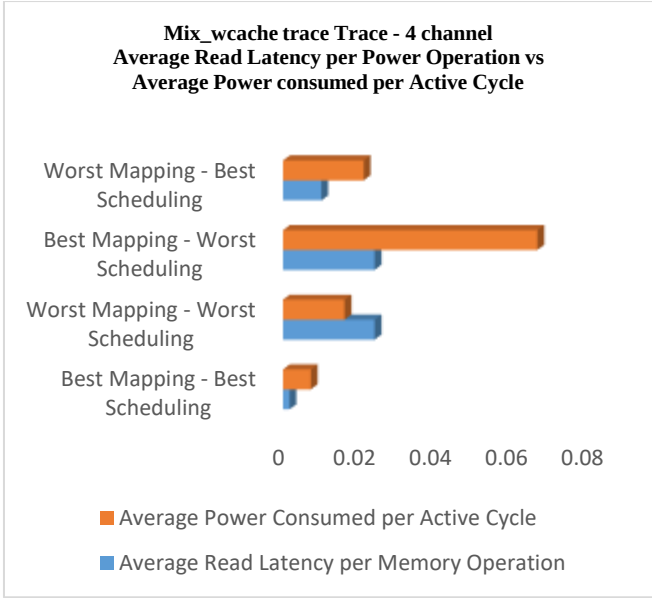


Fig. 10. ARLMO vs APAC mix_wcache trace plot (4-channel) .

(Results for the 1-channel configuration captured on Fig. 13, 14, 15, 16 and 17) can be found in the Appendix section).

A speedup trend plot and power plot between the BB and other mapping-scheduling pairings (WW, BW, WB) is shown on Fig. 11 and 12. The stream trace and mix_wcache traces had the highest speed-up gains among the memory traces. Traces (stream and mix_wcache) which recorded high speed-ups correspondingly had low-to-optimal APAC values on the APAC power plot. This clearly depicts that a good choice of address mapping scheme and scheduling can both hugely impact DRAM memory performance and power efficiency.

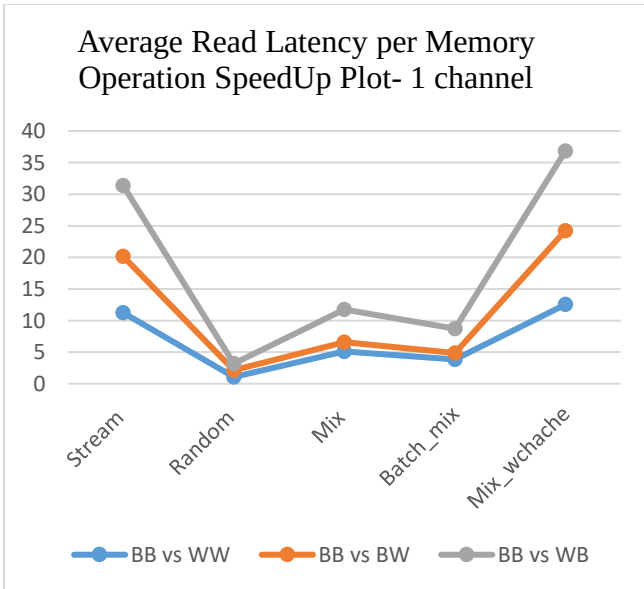


Fig. 11. ARLMO speedup plot 1-channel configuration.

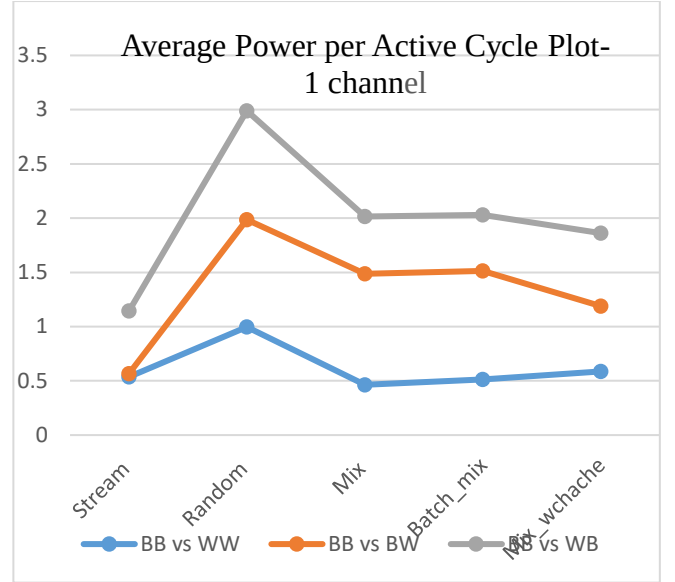


Fig. 12. APAC power plot for 1-channel configuration.

V. CONCLUSION AND FUTURE WORK

A. Recorded Data And Observations

This thesis project proposed a first order approximation power and performance model in terms of Average Read Latency per Memory Operation (ARLMO) and Average Power consumed per Active Cycle (APAC) for DRAM memory controllers. This proposed model was used to investigate the collective impact of matching both the address mapping scheme and scheduling policy to system-level applications on overall DRAM power and performance. It was shown that both power and performance (P&P) can be improved by maintaining a “better” DRAM controller mapping and scheduling policy settings based on memory requests stream. For each trace instance, an appropriate choice of both the address mapping and scheduling policy simulated (BB pairing), resulted in lower memory request execution latency. Also, overall DRAM power efficiency was greatly improved.

B. Future Work

The proposed power and performance model was evaluated using the DDR4 DRAM protocol (1 and 4 channel configurations) in this research study. Though the proposed model can be applicable across the various DRAM protocols memory controllers like (GDDR5, LPDDR4, WIO), this work limited its testing to only DDR4 DRAM memory controller. As a result, the proposed power and performance model should be extended to other DRAM protocols other than the DDR4 to achieve a generalized protocol-independent power and performance model. Also, the combinations of scheduling policies and address mapping schemes can be expanded to increase scope and adaptability of the proposed power and performance model.

Thus, this study should serve as a fundamental reference point for other DRAM (other than DDR4) memory power and performance studies.

ACKNOWLEDGMENT

This work was supported by the publications and materials, repositories referenced, and it is purely for academic purposes.

Special thanks to the advisory committee Dr. McLauchlan Lifford, Dr. Hicks David, Dr. Rahman Ashfaqur and Dr. Nekovei Reza for the support and guidance.

REFERENCES

- [1] C. Carvalho, "The gap between processor and memory speeds," in Proceedings of 3rd. Internal Conference on Computer Architecture, Braga, Portugal, 2002.
- [2] P. Machanick, "Approaches to addressing the memory wall," University of Queensland Brisbane, Australia, 2002.
- [3] P. Kollig, C. Osborne and T. Henriksson, "Heterogeneous multi-core platform for consumer multimedia applications," in 2009 Design, Automation & Test in Europe Conference & Exhibition, Nice, 2009.
- [4] Texas Instruments, "DDR3 Design Requirements for KeyStone Devices, Application Report," August 2011–Revised January 2018.
- [5] B. Jacob et al, "Memory systems: cache, DRAM, disk," Morgan Kaufmann Publishers, 2007.
- [6] O. Mutlu and T. Moscibroda, "Parallelism-Aware Batch Scheduling: Enabling High-Performance and Fair Shared Memory Controllers," IEEE Micro, 2009.
- [7] S. Goossens et al., "Conservative open-page policy for mixed time criticality memory controllers," in 2013 Design, Automation & Test in Europe Conference & Exhibition, Grenoble, France, 2013.
- [8] M. Paolieri et al., "Timing effects of DDR memory systems in hard real-time multicore architectures: Issues and solutions," ACM Transactions on Embedded Computing Systems, vol. 12, 2013.
- [9] B. Akesson et al, "Real-Time Scheduling Using Credit-Controlled Static-Priority Arbitration," in 2008 14th IEEE International Conference on Embedded and Real-Time Computing Systems and Applications, Kaohsiung, 2008.
- [10] B. Diniz, D. Guedes, J. Wagner Meira, and R. Bian, "Limiting the Power Consumption of Main Memory," in Proceedings of the 34th Annual International Symposium on Computer, 2007.
- [11] I. Hur, and C. Lin, "A Comprehensive Approach to DRAM Power Management," in IEEE 14th International Symposium on High Performance Computer Architecture, pages 305-316,, 2008.
- [12] S. C. J. K. a. C. C. Andra, "Memory Systems: Overview and Trends," Moritz Laboratory of Software Systems and Architectures Dept. of Electrical and Computer Engineering University of Massachusetts Amherst, Massachusetts, vol. 01002.
- [13] H. Hassan, G. Pekhimenko, N. Vijaykumarand, and O., "Chargecache: Reducing DRAM latency by exploiting row access locality," in Proceedings of 2016 IEEE International Symposium on High Performance Computer Architecture, HPCA, 2016.
- [14] S. Park and I. Park, "History-based memory mode prediction for improving memory performance," in Proceedings of IEEE International Symposium on Circuits and Systems, ISCAS, May 2003.
- [15] M. Ghasempour, A. Jaleel, J. D. Garside, and M. Lu, "Happy: Hybrid address-based page policy in DRAMS," in Proceedings of the Second International Symposium on Memory Systems, MEMSYS, 2016.
- [16] Tareq A. Alawneh, "A Dynamic Row-Buffer Management Policy for Multimedia Applications," in 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP), 2019.
- [17] Lixin Zhang et al., "The Impulse memory controller," IEEE Transactions on Computers, 2001.
- [18] C. Li et al, "Self-Adaptive Address Mapping Mechanism for Access Pattern Awareness on DRAM," in 2019 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCLOUD/SocialCom/SustainCom), Xiamen, China, 2019.
- [19] P. Pessl et al., "DRAMA: Exploiting DRAM Addressing," in Proceedings of the 25th USENIX Security Symposium, Austin, Texas, 2016.
- [20] Zhao Zhang, Zhichun Zhu and Xiaodong Zhang, "A permutation-based page interleaving scheme to reduce row-buffer conflicts and exploit data locality," in Proceedings 33rd Annual IEEE/ACM International Symposium on Microarchitecture, Monterey, CA, USA, 2000.
- [21] S. Rixner et al., "Memory access scheduling," in Proceedings of 27th International Symposium on Computer Architecture (IEEE Cat. No.RS00201), Vancouver, BC, Canada, 2000.
- [22] R. Ausavarungrun et al., "Staged memory scheduling: Achieving High Performance and Scalability in Heterogeneous Systems," in The 39th Annual International Symposium on Computer Architecture, 2012.
- [23] S. Bayliss and G. A. Constantinides, "Methodology for designing statically scheduled application-specific SDRAM controllers using constrained local search," in 2009 International Conference on Field-Programmable Technology, Sydney, NSW, 2009.
- [24] B. Akesson and K. Goossens, "Architectures and modeling of predictable memory controllers for improved system integration," 2011 Design, Automation & Test, Europe, Grenoble, 2011.
- [25] Ibrahim Hur and Calvin Lin, "Memory scheduling for modern microprocessors," ACM Transactions on Computer Systems, 2007.
- [26] J. Shao and B. T. Davis, "A Burst Scheduling Access Reordering Mechanism," in 2007 IEEE 13th International Symposium on High Performance Computer Architecture, Scottsdale, AZ, 2007.
- [27] C. Macian, S. Dharmapurikar and J. Lockwood, "Beyond performance: secure and fair memory management for multiple systems on a chip," in Proceedings 2003 IEEE International

Conference on Field-Programmable Technology (FPT) (IEEE Cat. No.03EX798), Tokyo, Japan, 2003.

- [28] Kun-Bin Lee, Tzu-Chieh Lin and Chein-Wei Jen, An efficient quality-aware memory controller for multimedia platform SoC, IEEE Transactions on Circuits and Systems for Video Technology, May, 2005.
- [29] S. Heithecker and R. Ernst, "Traffic shaping for an FPGA based SDRAM controller with complex QoS requirements," in Proceedings. 42nd Design Automation Conference, Anaheim, CA, 2005.
- [30] A. Burchardt, E. Hekstra-Nowacka and A. Chauhan, "A real-time streaming memory controller," Design, Automation and Test in Europe, Munich, Germany, 2005.
- [31] R. Jadaa, "The Performance And Power Impact Of Using Multiple Dram Address Mapping Schemes In Multicore Processors," University of Central Florida, 2011 .
- [32] Aastha Modgila et al., "Understanding and Analyzing the Impact of Memory Controller's Scheduling Policies on DRAM's Energy and Performance," in Proceedings of the 4th International Conference on Eco-friendly Computing and Communication Systems, 2015.
- [33] S. Srinivasan et al., "CMP Memory Modeling: How Much Does Accuracy Matter?," Systems Technology Lab, Intel Corporation, University of Maryland, College Park, 2005.
- [34] M. T. Yourst, "PTLsim: A Cycle Accurate Full System x86-64 Microarchitectural Simulator," in IEEE International Symposium on Performance Analysis of Systems & Software, San Jose, CA, 2007.
- [35] Micron Technology, "Calculating Memory System Power for DDR," Technical Report, 2001.
- [36] A. Hansson, N. Agarwal et al., "Simulating DRAM controllers for future system architecture exploration," in 2014 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), Monterey, CA, USA, 2014.
- [37] P. Rosenfeld, E Cooper-Balis and B. Jacob, "DRAMSim2: A Cycle Accurate Memory System Simulator," IEEE Computer Architecture Letters, 2011.
- [38] S. Li, Z. Yang et al., "DRAMsim3: A Cycle-Accurate, Thermal-Capable DRAM simulator," IEEE COMPUTER ARCHITECTURE LETTERS, JULY-DECEMBER 2020.
- [39] Shang Li, Dhiraj Reddy, and Bruce Jacob, "A Performance & Power Comparison of Modern High-Speed DRAM Architectures," in The International Symposium on Memory Systems (MEMSYS), Old Town Alexandria, VA, USA, 2018.
- [40] Micron Technology, "DDR4 SDRAM Datasheet 16Gb: x4, x8, x16 DDR4 SDRAM Features," Technical Report, 2018.
- [41] W.D. Weber, "Efficient Shared DRAM Subsystems for SOCs," in Microprocessor Forum, 2003.

APPENDIX

TABLE X. STREAM TRACE FILE VS ADDRESS MAPPING SCHEMES.

Address Mapping	Average Read Request Latency (cycles)	Average Power (mW)	ARLMO	APAC (mW/cycle)
Scheme A	21.24	743.71	0.08496	1.822818627
Scheme B	238.545	555.6	1.5903	1.578409091
Scheme C	235.273	555.6	1.176365	1.578409091
Scheme D	43.7	956.544	0.1748	1.990726327
Scheme E	223.273	555.6	0.893092	1.578409091
Scheme F	47.78	1056.48	0.1911	3.001363636

TABLE XI. RANDOM TRACE FILE VS ADDRESS MAPPING SCHEME.

Address Mapping	Average Read Request Latency (cycles)	Average power (mW)	ARLMO	APAC (mW/cycle)
Scheme A	45.53	1095.02	0.18212	2.278917794
Scheme B	46.53	1092.14	0.18612	2.321232731
Scheme C	47.31	1095.74	0.18924	2.268612836
Scheme D	45.89	1095.74	0.18356	2.273319502
Scheme E	47.33	1095.02	0.18932	2.278917794
Scheme F	49.33	1094.3	0.19732	2.289330544

TABLE XII. MIX TRACE FILE VS ADDRESS MAPPING SCHEME.

Address Mapping	Average Read Request Latency (cycles)	Average power (mW)	ARLMO	APAC (mW/cycle)
Scheme A	68.24	925.3	0.27296	1.925702393
Scheme B	112.14	753.22	0.44856	1.567575442

Scheme C	113.619	751.344	0.454476	1.585113924
Scheme D	48.52	1111.49	0.19408	2.320438413
Scheme E	94.1	765.65	0.3764	1.577033986
Scheme F	60.08	1011.26	0.24032	2.111189979

TABLE XIII. BATCH_MIX TRACE FILE VS ADDRESS MAPPING SCHEME.

Address Mapping	Average Read Request Latency (cycles)	Average power (mW)	ARLMO	APAC (mW/cycle)
Scheme A	64.28	981.02	0.36731	1.98989858
Scheme B	112.19	852.29	0.64108	1.771912682
Scheme C	111.26	852.86	0.63577	1.765755694
Scheme D	63.05	1092.19	0.36028	2.219898374
Scheme E	108.94	877.3	0.62251	1.784944049
Scheme F	59.48	1092.48	0.33988	2.215983773

TABLE XIV. MIX_WCACHE TRACE FILE VS ADDRESS MAPPING SCHEME.

Address Mapping	Average Read request Latency (cycles)	Average power (mW)	ARLMO	APAC (mW/cycle)
Scheme A	36.3	969.46	0.1452	1.996828012
Scheme B	113.91	772.9	0.45564	1.635767196
Scheme C	113.61	774.77	0.45444	1.617473904
Scheme D	44.97	1099.2	0.17988	2.374082073
Scheme E	105.83	785.62	0.42332	1.722850877
Scheme F	44.37	1096.03	0.17748	2.424845133

TABLE XV. STREAM TRACE VS SCHEDULING POLICY.

Scheduling policy	Average Read request Latency (cycles)	Average power (mW)	Average Read Latency per Memory Operation	Average Power per Active cycle (mW/cycle)
Open page	21.24	743.71	0.085	1.823
Close page	720.297	423.54	2.881	1.303
Open page variant	21.24	738.528	0.143235	1.678
Close page variant	223.27	525.5	0.893	1.635

TABLE XVI. RANDOM TRACE VS SCHEDULING POLICY.

Scheduling policy	Average Read request Latency (cycles)	Average power (mW)	Average Read Latency per Memory Operation	Average Power per Active cycle (mW/cycle)
Open page	52.7105	1093.54	0.263552	2.25
Close page	40.06	423.474	0.160	1.478
Open page variant	45.53	1095.02	0.182	2.279
Close page variant	39.89	1034.54	0.159	3.824

TABLE XVII. MIX TRACE VS SCHEDULING POLICY.

Scheduling policy	Average Read request Latency (cycles)	Average power (mW)	Average Read Latency per Memory Operation	Average Power per Active cycle (mW/cycle)
Open page	68.24	925.3	0.27296	1.926
Close page	148.24	423.575	0.59296	1.486
Open page variant	75.16	925.3	0.30064	1.926
Close page variant	109.67	649.68	0.43868	5.369

TABLE XVIII. BATCH MIX TRACE TRACE VS SCHEDULING POLICY.

Scheduling policy	Average Read request Latency (cycles)	Average power (mW)	Average Read Latency per Memory Operation	Average Power per Active cycle (mW/cycle)
Open page	64.28	981.024	0.367	1.99
Close page	220.075	423.451	1.258	1.518
Open page variant	59.275	981.024	0.339	1.99
Close page variant	153.129	764.448	0.875	4.343

TABLE XIX. MIX_WCACHE TRACE TRACE VS SCHEDULING POLICY.

Scheduling policy	Average Read request Latency (cycles)	Average power (mW)	Average Read Latency per Memory Operation	Average Power per Active cycle (mW/cycle)
Open page	36.3	969.46	0.145	2.0176
Close page	166.267	423.53	0.665	1.51
Open page variant	36.3	969.46	0.145	2.0176
Close page variant	107.08	688.13	0.428	4.915

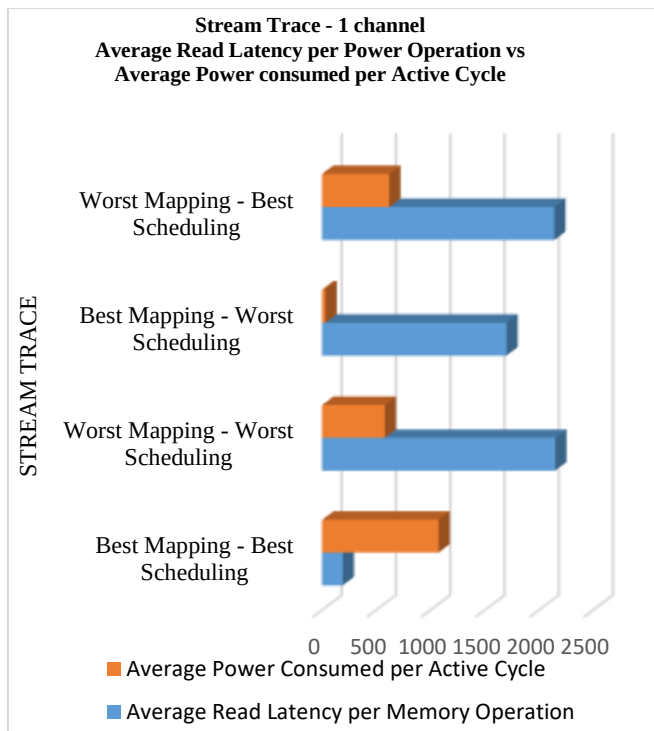


Fig. 13. ARLMO vs APAC stream trace plot (1 -channel).

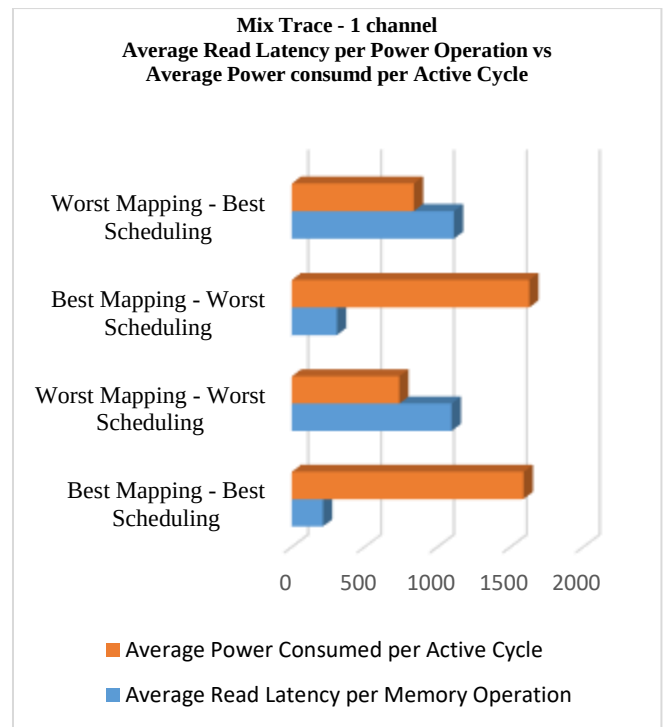


Fig. 15. ARLMO vs APAC mix trace plot (1 -channel).

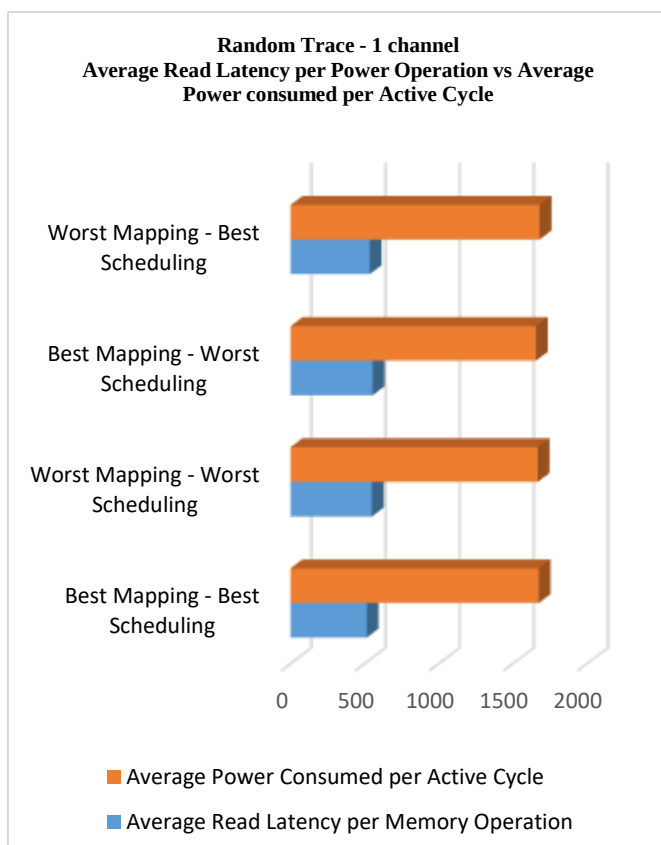


Fig. 14. ARLMO vs APAC random trace plot (1 -channel).

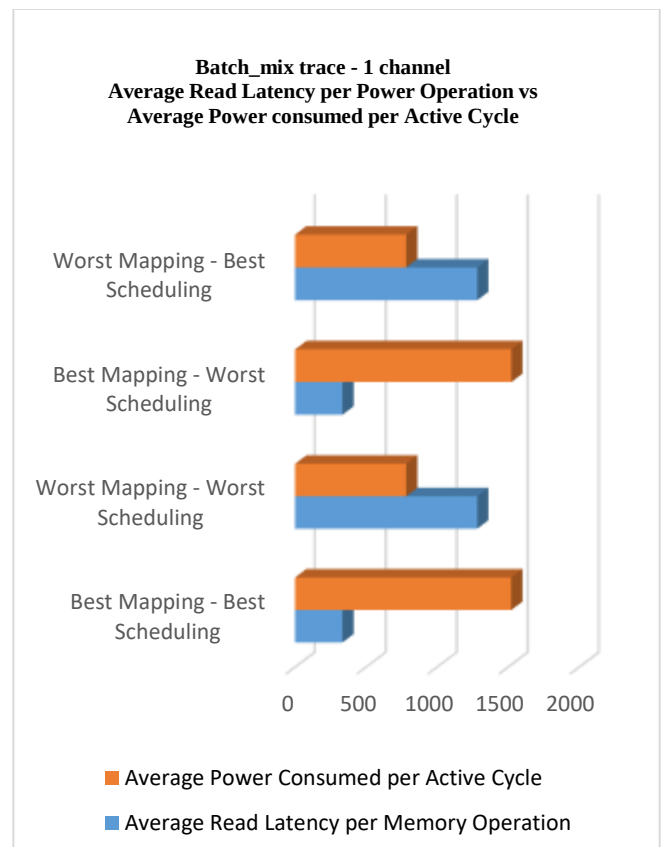


Fig. 16. ARLMO vs APAC batch_mix trace plot (1 -channel).

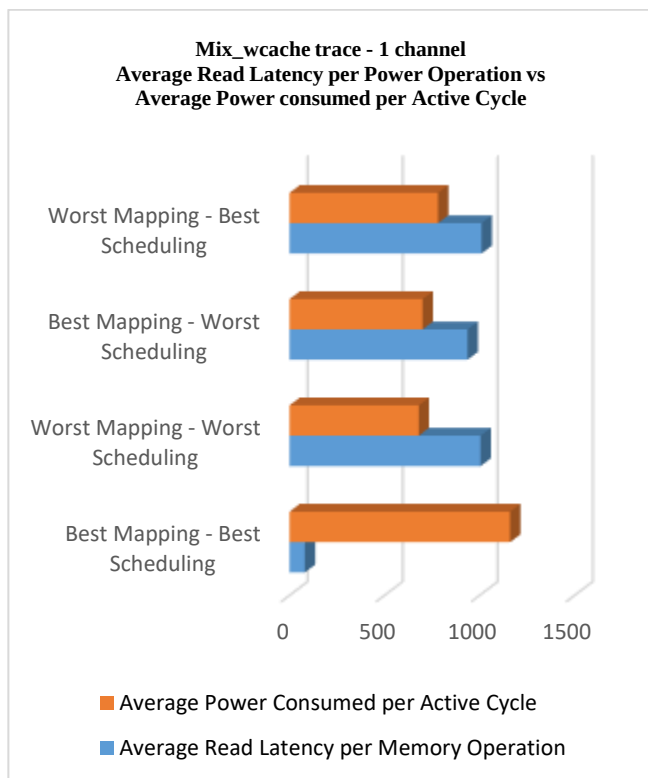


Fig. 17. ARLMO vs APAC mix_wcache trace plot (1 -channel).